
Contents

1	Introduction	2
2	Summary of Preparatory Tasks	2
3	Task 1.1: Combinational Adder Implementation	3
3.1	Objective	3
3.2	Lab Setup	3
3.3	Test-Table for Combinational Adder	4
3.4	Oscilloscope Results	5
3.5	Conclusion	5
4	Task 1.2: Registered Adder with Synchronous Control	5
4.1	Objective	5
4.2	Lab Setup	6
4.3	Oscilloscope Results	6
5	Task 1.3: Implementation of an Accumulator	7
5.1	Objective	7
5.2	Lab Setup	7
5.3	Accumulator Sequence Visualized	8
5.4	Conclusion	8
6	Task 2.1: Accumulator as Counter	9
6.1	Objective	9
6.2	Lab Setup	9
7	Task 2.2: Counter with Internal Feedback	13
7.1	Objective	13
7.2	Lab Setup	13
7.3	Testing Procedure	14
7.4	Conclusion	16
8	Literature and References	17
9	Appendix	17

1 Introduction

In this lab, we worked on designing and testing an 8-bit arithmetic circuit using VHDL on an FPGA platform. Our goal was to better understand how combinational and sequential digital circuits behave, especially when applied to real-world tasks like counting or accumulating values over time.

The lab was split into three main parts:

- First, we built a basic combinational adder and checked how it handles inputs and carries.
- Then, we added registers to the design to store results in sync with a clock.
- Finally, we turned the adder into an accumulator by feeding its output back to its input.

Throughout the lab, we used Vivado to synthesize and load our designs onto the FPGA board, and EDAPlayground for simulation. We also used IOM boards for input/output control and an oscilloscope to observe signal timing.

2 Summary of Preparatory Tasks

Task 1.1: Combinational Adder

An 8-bit combinational adder was implemented to compute the equation $Y = A + B + C_{in}$, where A and B are 8-bit inputs and C_{in} is the carry-in bit. The output Y is the 8-bit result, and a separate C_{out} represents the carry-out. A complete testbench was developed using EDAPlayground to simulate and verify the circuit. A total of 20 meaningful test vectors were written to cover normal additions, carry-in influence, overflow behavior, and edge cases such as maximum input values (255 + 255).

Task 1.2: Registered Adder

The basic adder from Task 1.1 was extended by introducing clocked registers for Y and C_{out}, creating a sequential design. The registered adder updates its output only on the rising edge of the clock and only when the active-high enable signal (en) is asserted. A synchronous, active-low reset (resetn) was added to clear the output to zero when needed. The new set of 20 test vectors was used in a new testbench, now including clock generation, reset handling, and enable logic. Simulation verified correct timing behavior, conditional output update, and reset functionality.

Task 1.3: I/O Configuration

Three constraint files were used: Adder IOM Conn 3.xdc, Adder IOM Conn 4.xdc, and Modsys Artix7 Main System.xdc. Switches on two IOM boards were connected to operand A and operand B. Keys were used for carry-in and enable. Clock and reset came from the main board.

Task 2.1: Accumulator Setup

In this task, the registered adder was reconfigured to act as an accumulator by feeding back the output Y into the input A . The operand B was fixed at either `00000001` for up-counting or `11111111` for down-counting. On each clock cycle, the adder accumulated the result of the previous sum with the constant B . The design was simulated under different clocking conditions (manual, low-frequency, high-frequency), and propagation delay was measured using an oscilloscope to verify synchronous behavior.

Task 2.2: Internal Feedback

To make the accumulator fully self-contained, the feedback path from Y to A was implemented internally within the architecture. An internal signal `internal_a` was introduced to store the previous output and was updated only on rising clock edges when the enable signal was asserted. Unlike Task 2.1, the input ports A and B were left unused, and the adder became a self-updating register-based counter. This setup allowed the design to freeze its output when `en = 0` and resume accumulation when `en = 1`, demonstrating effective gating logic.

Task 2.3: Constraint File Adaptation

To support modified input behavior on the FPGA, the original `Adder_IOM_Conn_3.xdc` file was adapted. In the new version, the physical switches were remapped to control the `b_vector` input instead of `a_vector`.

3 Task 1.1: Combinational Adder Implementation

3.1 Objective

The objective of this task was to design and implement an 8-bit combinational adder in VHDL that computes the sum of two input vectors along with a carry-in signal. The circuit outputs an 8-bit result and a carry-out, using only combinational logic without any clocked elements. This task focused on understanding how addition is handled purely through logic gates and how carry propagation affects circuit behavior.

3.2 Lab Setup

The following configuration was used:

- **Input Configuration:** The 8-bit operands `a_vector` and `b_vector` were connected to switches on two separate IOM boards. The carry-in signal (`cin`) was assigned to a pushbutton key on one of the IOM boards.
- **Output Display:** The result vector Y and carry-out C_{out} were displayed using the onboard LEDs for visual verification.
- **Clock Source:** No clock was used.
- **Oscilloscope Monitoring:** The propagation delay from input to output was measured using two channels:

- **Channel 1:** Connected to the carry-in signal (C_{in}).
- **Channel 2:** Connected to one of the output signals C_{out} .

3.3 Test-Table for Combinational Adder

Req. ID	Test description	stimuli operand A	stimuli operand B	stimuli carry in	expected response Y	expected response carry out	Testbench Simulation (check)	FPGA circuit (check)
R.1.1	Basic addition	00000000	00000001	0	00000001	0	✓	✓
R.1.2	Basic addition	00000001	00000001	0	00000010	0	✓	✓
R.1.3	Basic addition	00000000	00000001	1	00000010	0	✓	✓
R.1.4	Basic addition	00000000	00000000	0	00000000	0	✓	✓
R.1.5	Basic addition	00000010	00000001	0	00000011	0	✓	✓
R.1.6	Basic addition	00000100	00000010	0	00000110	0	✓	✓
R.1.7	Basic addition	00000101	00000011	1	00000100	0	✓	✓
R.2.1	Boundary / overflow	11111111	00000000	1	00000000	1	✓	✓
R.2.2	Boundary / overflow	11111111	11111111	0	11111110	1	✓	✓
R.2.3	Boundary / overflow	11111111	11111111	1	11111111	1	✓	✓
R.2.4	Boundary / overflow	00000001	11111111	0	00000000	1	✓	✓
R.2.5	Boundary / overflow	00000001	11111111	1	00000001	1	✓	✓
R.2.6	Boundary / overflow	01111111	00000001	0	10000000	0	✓	✓
R.2.7	Boundary / overflow	01111111	00000001	1	10000001	0	✓	✓
R.3.1	Alternating Bit Patterns	11110000	00001111	0	11111111	0	✓	✓
R.3.2	Alternating Bit Patterns	11110000	00001111	1	00000000	1	✓	✓
R.3.3	Alternating Bit Patterns	10101010	01010101	0	11111111	0	✓	✓
R.3.4	Alternating Bit Patterns	10101010	01010101	1	00000000	1	✓	✓
R.4.1	Mid Range Sums	00001111	00001111	0	00011110	0	✓	✓
R.4.2	Mid Range Sums	00001111	00001111	1	00011111	0	✓	✓
R.4.3	Mid Range Sums	00000001	00000010	0	00000011	0	✓	✓

Figure 1: Original Test Table, Measured and Checked During the Lab Session

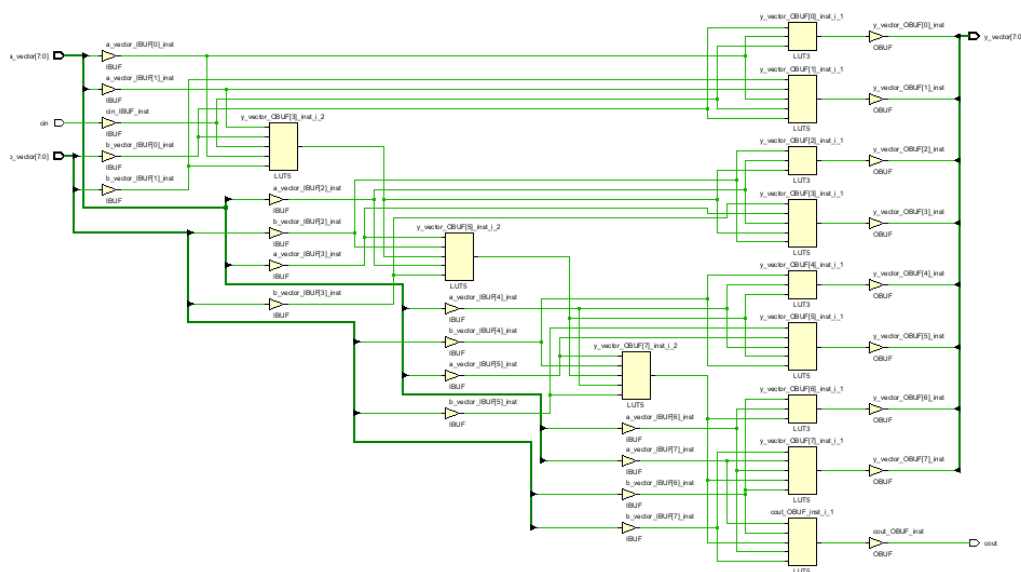


Figure 2: Technology Schematic of Combinational Adder

3.4 Oscilloscope Results

The waveform shows the timing behavior of the combinational adder in response to a change at one of its inputs. The delay between the input transition and the corresponding output response is measured to be approximately **7.6 ns**, as indicated by the cursor readings in the upper right corner. This reflects the propagation delay through the adder's logic path and confirms the expected delay introduced by carry signal propagation.

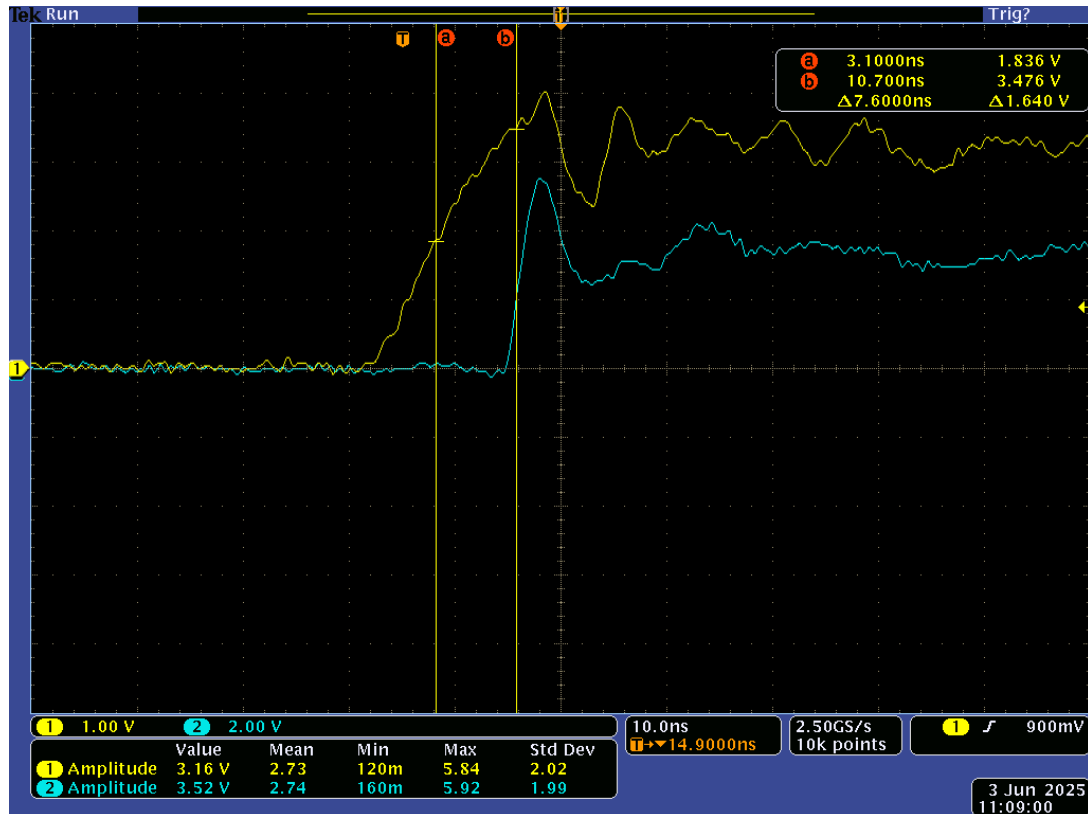


Figure 3: Propagation Delay

3.5 Conclusion

In Task 1.1, we designed an 8-bit combinational adder to understand how addition works using only logic gates, without any clock or memory. Implementing it in VHDL and testing it on the FPGA showed how outputs respond immediately to input changes, and how carry propagation affects timing. This task clarified the limitations of combinational logic and prepared us for working with sequential designs in the next step.

4 Task 1.2: Registered Adder with Synchronous Control

4.1 Objective

The goal of Task 1.2 was to modify the combinational adder by adding output registers. These registers store the result and carry-out only on the rising edge of the clock, controlled by an enable signal and a reset input. This turned the circuit into a sequential

design where the output updates only when allowed by the control signals. The design was implemented in Vivado, tested on the FPGA, and confirmed to behave as expected under manual and clocked operation.

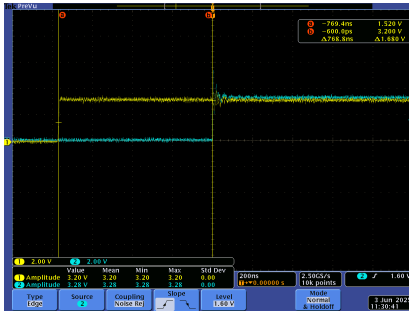
4.2 Lab Setup

The following configuration was used:

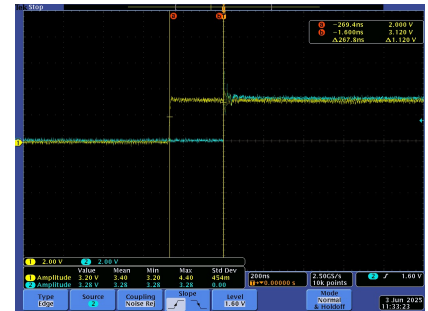
- **Input Configuration:** Inputs A , B , and C_{in} were connected to switches. Enable and RESETN signals were controlled through dedicated keys.
- **Output Display:** Outputs Y and C_{out} were monitored using the LED indicators.
- **Clock Source:** A manual clock (push-button) was used during initial testing. A low-frequency internal clock was also available for comparison.
- **Oscilloscope Monitoring:** The following connections were used to measure clock-to-output delay:
 - **Channel 1:** Connected to the clock signal.
 - **Channel 2:** Connected to one of the output bits (e.g., $Y(0)$) or the carry-out signal.

4.3 Oscilloscope Results

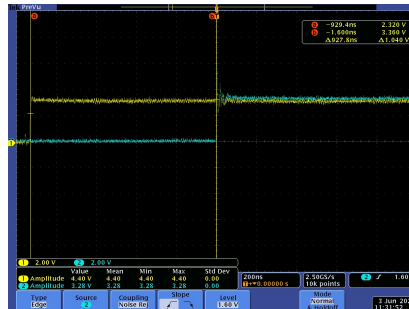
The waveforms show different delay between the rising clock edge (yellow) and the output change (blue). Since the circuit was tested with manual clocking, the delay varies depending on how fast the clock button is pressed. This variability is expected and reflects both the register response time and the manual input timing.



(a) Click 1 – 768.8ns



(b) Click 2 – 267.8ns



(c) Click 3 – 927.8ns

Figure 4: Oscilloscope Measurements During Registered Adder Operation

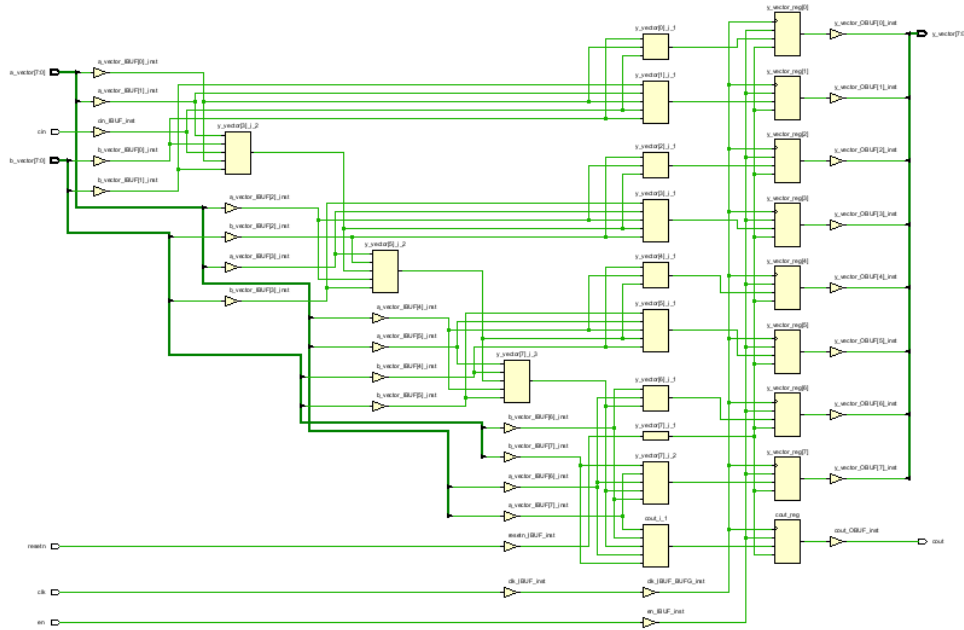


Figure 5: Technology Schematic of Registered Adder

5 Task 1.3: Implementation of an Accumulator

5.1 Objective

The goal of Task 1.3 was to turn the registered adder into an accumulator by connecting its output back to one of its inputs. This setup allowed the circuit to keep adding a new operand to the previous result on each clock cycle, simulating a basic up-counter controlled by manual input.

5.2 Lab Setup

The following configuration was used:

- **Operand A Re-configuration:** The output vector Y was physically wired back to input A , creating a feedback loop that enabled accumulation over multiple clock cycles.
- **B Input Configuration:** Operand B was set manually using switches for each addition step. The following values were used in sequence:
 - $B = 00000011$ (3)
 - $B = 00000101$ (5)
 - $B = 00001001$ (9)
 - $B = 00010111$ (23)
- **Clock Source:** A manual clock was used to trigger each accumulation step.
- **Output Display:** The results of each addition were monitored using the output LEDs.

- **Reset Functionality:** The RESETN key was used to clear the accumulator and restart the process.

5.3 Accumulator Sequence Visualized

We tested the accumulator by applying the values 3, 5, 9, and 23 one after the other using the switches. After each manual clock pulse, the current sum was fed back to the adder's input and combined with the next value. The results accumulated step by step as follows:

Step	Previous Y	Operand B	New Y
1	0	3	3
2	3	5	8
3	8	9	17
4	17	23	40

5.4 Conclusion

By wiring the output of the registered adder back into its input, we turned it into a working accumulator. Each new operand added to the stored result step by step, just like a basic counter. Using manual clock pulses made it easy to see how the values built up over time.

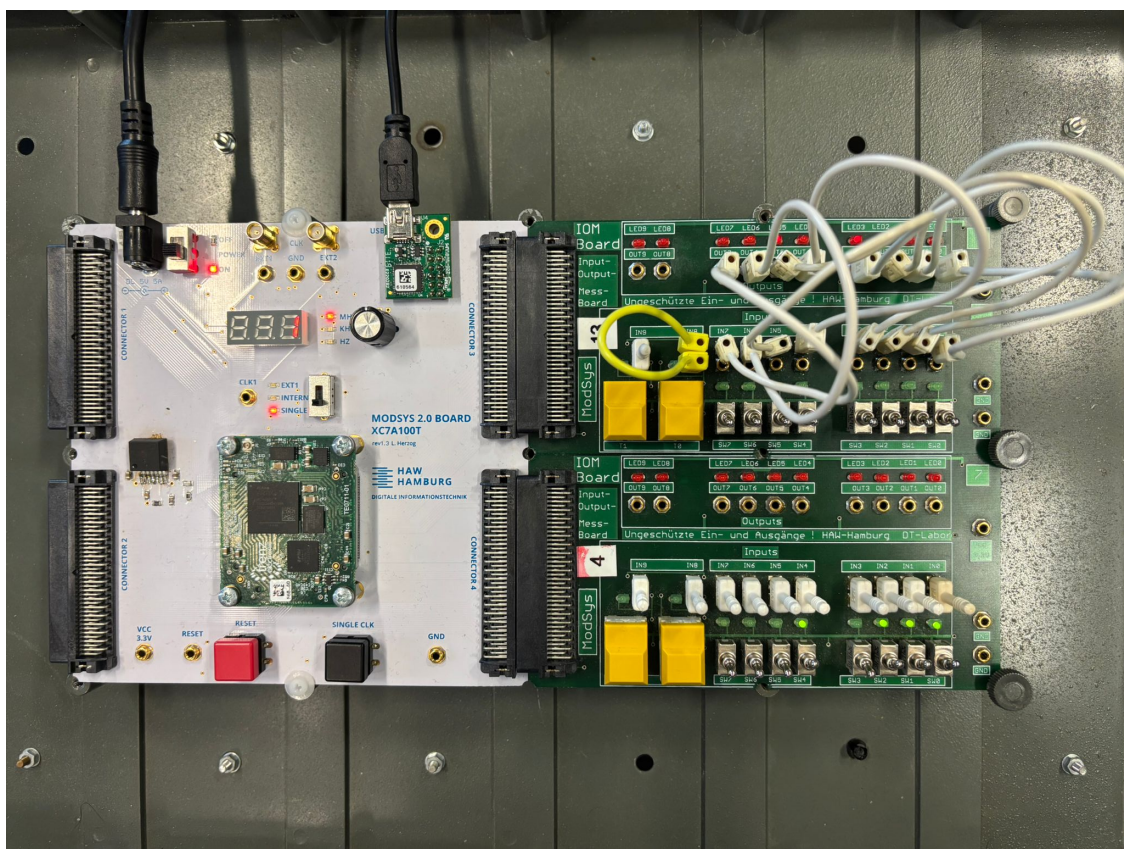


Figure 6: Board Setup for the Accumulator

6 Task 2.1: Accumulator as Counter

6.1 Objective

The objective of Task 2.1 is to utilize the previously implemented registered adder circuit to function as an accumulator, effectively transforming it into a simple digital counter. This is done by **feeding the output (`y_vector`) back into the input (`a_vector`)** on each clock cycle, while the second input (`b_vector`) is held constant at 00000001 (binary for decimal 1). The `cin` input is also fixed to '0' to simplify the logic and prevent any carry-in impact.

This setup creates either an up-counter or a down-counter depending on the value of `b_vector`. The circuit is tested using manual clock pulses, a low-frequency clock, and a high-frequency clock to observe how the count changes over time. We also measure the propagation delay between the clock input and the output change, focusing on the delay observed in the three least significant bits.

6.2 Lab Setup

The following configuration was used:

- **Operand A Re-configuration:** The operand A input of the registered adder was reconfigured to receive the previous output `Y` from the adder. This feedback loop allowed the adder to accumulate values over time, enabling it to function as a counter.
- **B Input Configuration:** The operand B was set to a constant value:
 - `B = 00000001` to make the accumulator count **up**.
 - `B = 11111111` to make it count **down**.
- **Clock Source:** Different tests were performed using manual pulses, low-frequency, and high-frequency clock signals.
- **Oscilloscope Monitoring:** The behavior of the output bits was observed using an oscilloscope. The connections were as follows:
 - **Channel 1:** Connected to the `clock` input.
 - **Channel 2:** Connected to the least significant bit of the output (`y_vector(0)`).
 - **Channel 3:** Connected to the second least significant bit (`y_vector(1)`).
 - **Channel 4:** Connected to the third least significant bit (`y_vector(2)`).

These channels allowed us to measure and analyze the propagation delay between the rising edge of the clock and the corresponding bit transitions in the output.

Clock Behavior and Timing Analysis

Manual Clocking The circuit was first tested using manual clock pulses triggered by a push button on the IOMI board. In this mode, each press generated a rising edge, causing the adder to perform a single accumulation step. Both **up-counting** and **down-counting** behavior were tested:

- With $B = 00000001$, the accumulator increased its value by 1 on every press.
- With $B = 11111111$, it decreased the value by 1 each time.

This setup allowed us to carefully observe and verify the functionality of the accumulator in both directions.

Low-Frequency Clock: The circuit was operated with a continuous but low-frequency clock signal. This allowed the output values to be clearly observed with enough time between transitions. The count could be followed visually on output LEDs. Both up and down counting were confirmed under these conditions, and the slower speed made it easier to measure bit-level timing.

High-Frequency Clock: Finally, a high-frequency was used to drive the counter. The accumulator continued to function correctly at this speed; however, the bit changes occurred so quickly that manual observation was no longer reliable. The output had to be monitored using an oscilloscope.

Propagation Delay: During all clocking tests, a delay was observed between the rising edge of the clock and the update of the output value `y_vector`. In this experiment, the delay between the clock edge and the response of the three least significant bits (`y_vector[0]`, `y_vector[1]`, `y_vector[2]`) was measured using the oscilloscope. The measured propagation delay was approximately **8.4 ns**, as shown in the waveform capture.

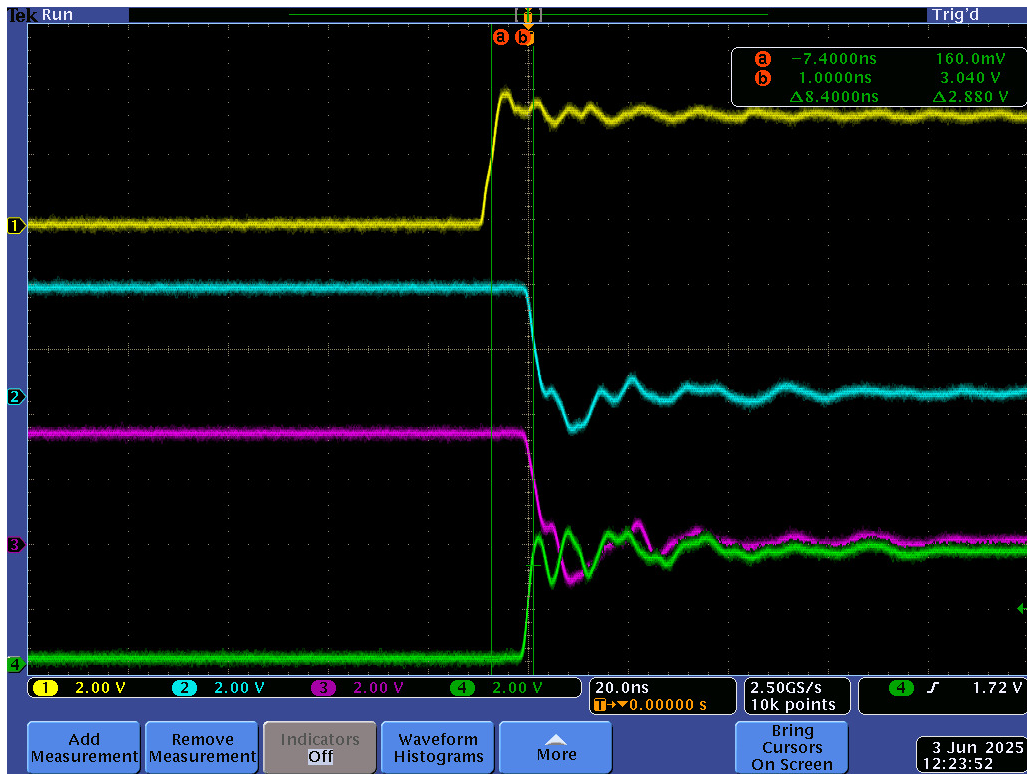


Figure 7: Waveform showing propagation delay between clock and y_vector output.

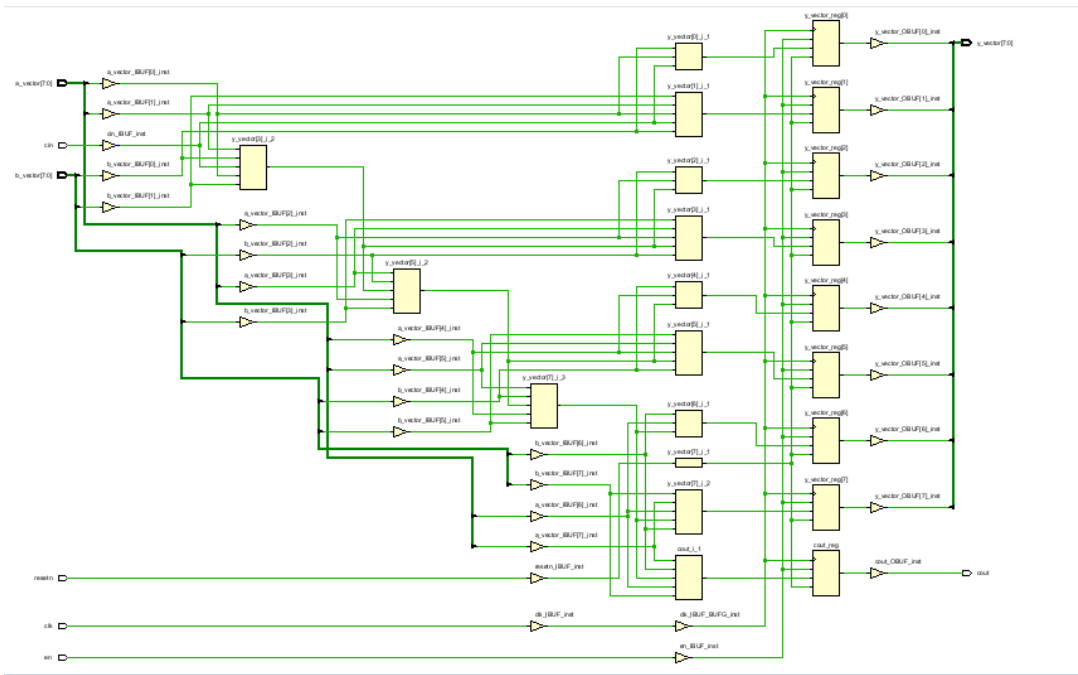


Figure 8: Schematic view of the registered adder design in Vivado. The design shows how the output y_vector is fed back to a_vector, enabling accumulator functionality.

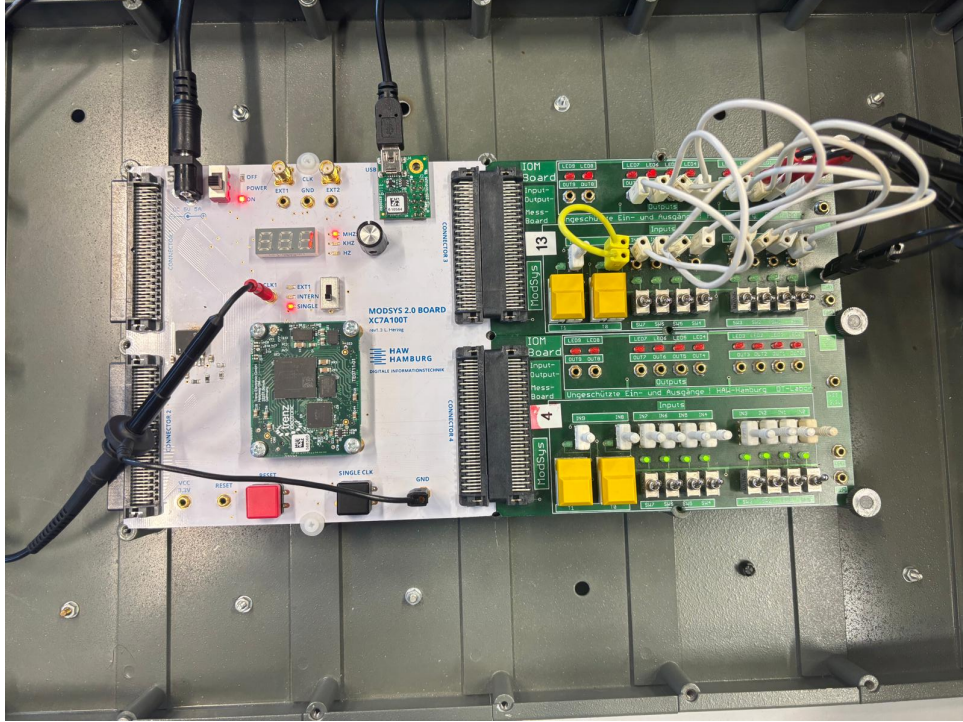


Figure 9: The circuit design

Conclusion

The registered adder was configured as an accumulator to function as a counter by externally feeding back the output `y_vector` into the input `a_vector` through the FPGA's I/O mappings. This physical loop was implemented via wiring, where the output displayed on LEDs was routed back as input via switches. This setup allowed accumulation without modifying the internal architecture.

The system was tested under manual, low-frequency, and high-frequency clocking conditions. Manual clocking allowed step-by-step validation of the counting behavior. At low frequencies, output transitions were clearly observable, while at high frequencies, the functionality was confirmed through oscilloscope analysis.

The measured propagation delay from the rising edge of the clock to the output update was approximately **8.4 ns**.

7 Task 2.2: Counter with Internal Feedback

7.1 Objective

The objective of Task 2.2 is to verify the behavior of the registered adder when the enable signal (`en`) is deactivated during operation. Specifically, the goal is to confirm that the output (`y_vector`) remains stable and does not update when `en = 0`, even though the clock continues running and external input values such as `b_vector` and `cin` may still be changing.

In this experiment, the accumulator is initially allowed to operate for several clock cycles with `en` set to 1, during which the adder performs regular accumulation and updates its output. Afterwards, the enable signal is set to 0 to disable further updates. The expected behavior is that the output freezes and remains unchanged for the remainder of the simulation, regardless of ongoing clock activity or external input changes.

To support this functionality, the architecture was modified as follows:

- A new internal signal, `internal_a`, was introduced to replace direct usage of `a_vector`.
- `internal_a` is initialized to zero and updated with the value of `y_vector` after each valid accumulation cycle.

This design ensures that `internal_a` updates only on rising clock edges when `en = 1`. When the enable signal is low, the internal state remains unchanged, effectively holding the current output value. This confirms that the design correctly implements enable-based gating and can selectively pause accumulation without data corruption or unintended behavior.

7.2 Lab Setup

The experiment was conducted on the FPGA and IOMI boards, using the existing on-board clock source as the clock input for the circuit.

The oscilloscope was connected as follows:

- Channel 1: connected to the clock input.
- Channel 2: connected to bit 0 (LSB) of `y_vector`.
- Channel 3: connected to bit 1 of `y_vector`.
- Channel 4: connected to bit 2 of `y_vector`.

Unlike Task 2.1, where the counter used `b_vector` as a fixed increment or decrement value, in Task 2.2, `b_vector` is not used at all in the actual circuit logic.

The only operand used for accumulation is `a_vector`, which is implemented internally as a feedback from the previous output (`y_vector`).

7.3 Testing Procedure

To verify the functionality and timing behavior of the accumulator in Task 2.2, several test cases were performed under different clocking conditions. The focus was on how the circuit responds when the enable signal (`en`) is either active or inactive, and on measuring the timing characteristics of the output.

- **Manual Clocking:** The circuit was first tested using manual clock pulses. A button was used to trigger a rising clock edge, while a second button was used to temporarily set `en = 1` during the pulse. This setup observation of the accumulator's behavior. When `en = 1`, the output incremented with each clock pulse. When `en = 0`, the output remained unchanged, confirming proper enable handling.
- **Low-Frequency Clock:** Next, a low-frequency internal clock was used. This made the output transitions slow enough to observe clearly on the oscilloscope. The accumulator behavior was tested both with `en = 1` (where normal accumulation occurred) and `en = 0` (where the output was held constant).
- **High-Frequency Clock:** The clock frequency was increased to observe the design's behavior at higher speeds. Although transitions became too fast to detect with the naked eye, they were still observable on the oscilloscope. The circuit correctly updated its output when `en = 1`, and held the previous value when `en = 0`.
- **Propagation Delay:** The propagation delay between the rising clock edge and the corresponding output change was measured using the oscilloscope. Channel 1 was connected to the clock signal, while Channels 2–4 were connected to the three least significant bits of `y_vector`. The oscilloscope trigger was set on the most significant bit being monitored to precisely capture the output transition.

The delay between the rising clock edge and the update of the output signal was **8.2 ns**.

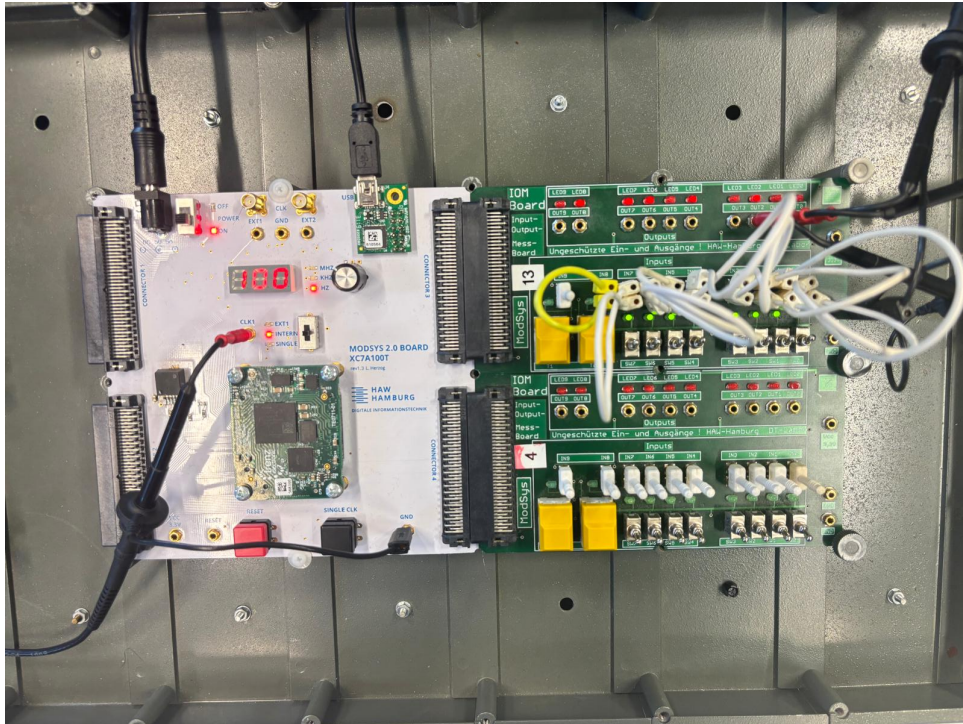


Figure 10: The circuit design

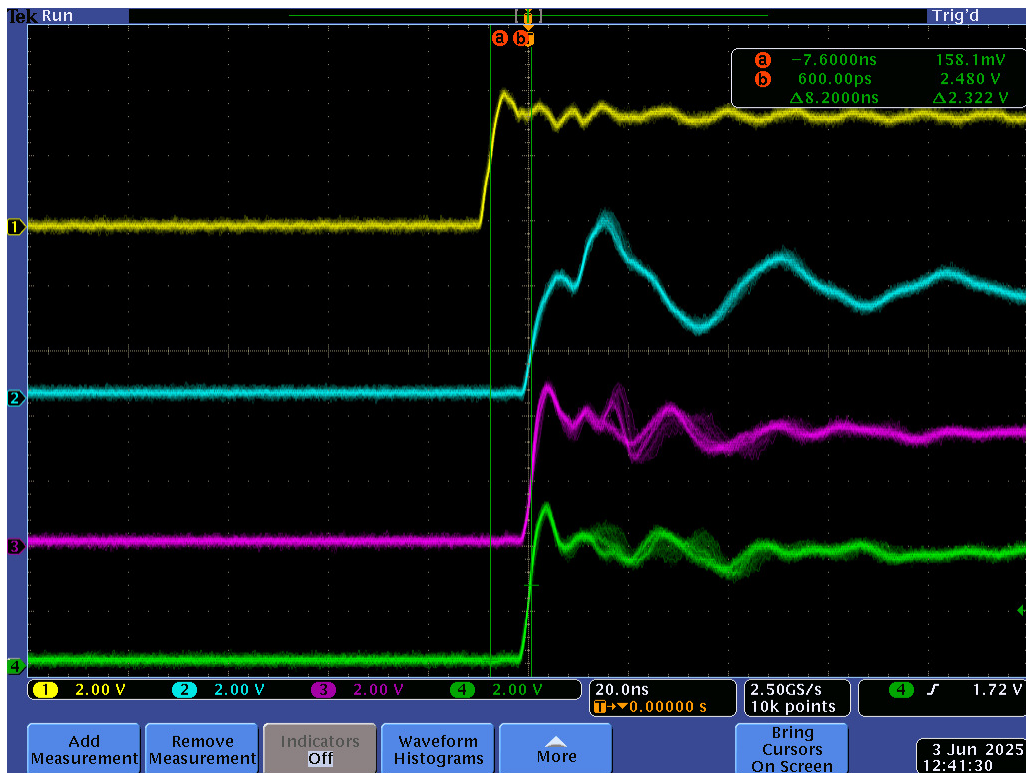


Figure 11: Propagation delay measured at 100 Hz clock frequency

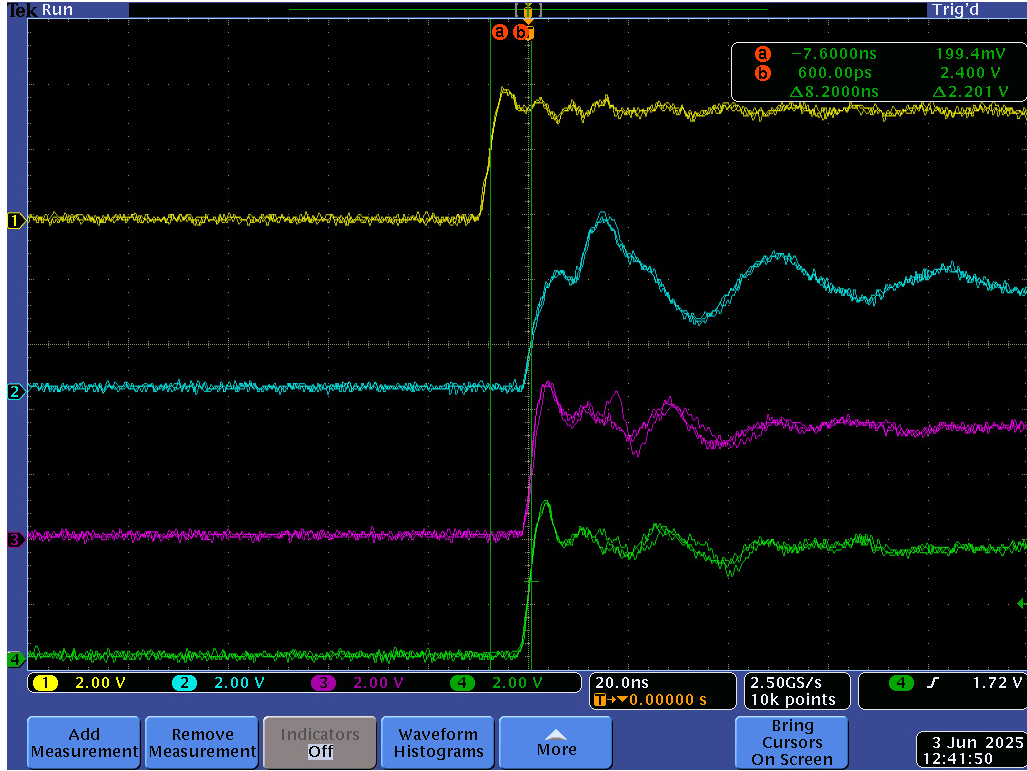


Figure 12: Propagation delay measured at 100 MHz clock frequency

7.4 Conclusion

The behavior of the registered accumulator was tested with a focus on the effect of the enable signal `en`. The design was configured so that the output `y_vector` was internally fed back into the accumulator, with no use of the external `b_vector` input.

The circuit was tested under various conditions, including manual clocking, low-frequency, and high-frequency operation. When `en` = 1, the accumulator updated its output normally on each rising clock edge. When `en` = 0, the output remained constant, confirming that the enable logic correctly prevents further accumulation.

Oscilloscope measurements confirmed that no changes occurred on the output while the `en` was deactivated, despite ongoing clock signals. This verified that the accumulator responds to clock edges when `en` = 1.

8 Literature and References

References

- [1] EDA Playground, *Online VHDL/Verilog Simulator*, Available at: <https://www.edaplayground.com>, Accessed June 2025.
- [2] Xilinx, *Vivado Design Suite User Guide*, Xilinx Inc., Available at: <https://www.xilinx.com/products/design-tools/vivado.html>, Accessed June 2025.
- [3] Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with VHDL Design*, 3rd ed., McGraw-Hill, 2009.
- [4] M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed., Pearson, 2017.
- [5] C.-H. Chang, J. Gu, and M. Zhang, “A review of 0.18- μ m full-adder performances for tree-structured arithmetic circuits,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, Vol. 13, No. 6, 2005.

9 Appendix

The following Design codes were used in the FPGA implementation and in Vivado simulations to define pin assignments and clock settings.

```
-- Code your design here
library IEEE;
use IEEE.std_logic_1164.all;

--use IEEE.numeric_std.all;

entity adder is
  generic (N: natural := 8);
  port(  a_vector: in std_logic_vector(N-1 downto 0);
        b_vector: in std_logic_vector(N-1 downto 0);
        y_vector: out std_logic_vector(N-1 downto 0);
        cin: in std_logic;
        cout: out std_logic);
end entity;

architecture for_loop of adder is
begin
  adding: process(all) -- requires VHDL-2008 setting in Vivado
    variable temp: std_logic;
    begin
      y_vector <= (others => '0'); -- default assignment
      temp := cin;
      repeat:
```

```

        for I in 0 to N-1 loop
            y_vector(I) <= a_vector(I) xor b_vector(I) xor temp;
            temp := (a_vector(I) and b_vector(I)) or (a_vector(I)
                and temp) or (b_vector(I) and temp);
        end loop;
        cout <= temp;
    end process;
end architecture for_loop;

-- Code your design here
library IEEE;
use IEEE.std_logic_1164.all;
--use IEEE.numeric_std.all;

entity registered_adder is
    generic (N: natural := 8);
    port( a_vector: in std_logic_vector(N-1 downto 0);
          b_vector: in std_logic_vector(N-1 downto 0);
          y_vector: out std_logic_vector(N-1 downto 0);
          cin: in std_logic;
          cout: out std_logic;
          clk: in std_logic;
          resetn: in std_logic;
          en: in std_logic);
end entity;
-----
architecture for_loop of registered_adder is
begin
    adding: process(clk)
        variable temp: std_logic;
    begin
        if rising_edge(clk) then
            if resetn = '0' then -- TODO: replace 'x' by the correct value
                y_vector <= (others => '0');
                cout <= '0'; -- fixed December 2024
            elsif en = '1' then -- TODO: replace 'x' by the correct value
                temp := cin;
                repeat:
                    for I in 0 to N-1 loop
                        y_vector(I) <= a_vector(I) xor b_vector(I) xor temp;
                        temp := (a_vector(I) and b_vector(I)) or (a_vector(I)
                            and temp) or (b_vector(I) and temp);
                    end loop;
                    cout <= temp;
                end if;
            end if;
        end process;
    end adding;
end architecture for_loop;

```

```

end architecture for_loop;

-- Code your design here
library IEEE;
use IEEE.std_logic_1164.all;
--use IEEE.numeric_std.all;

entity registered_adder is
  generic (N: natural := 8);
  port(  a_vector: in std_logic_vector(N-1 downto 0);
        b_vector: in std_logic_vector(N-1 downto 0);
        y_vector: out std_logic_vector(N-1 downto 0);
        cin: in std_logic;
        cout: out std_logic;
        clk: in std_logic;
        resetn: in std_logic;
        en: in std_logic);
end entity;
-----
architecture for_loop of registered_adder is
begin
  adding: process(clk)
    variable temp: std_logic;
  begin
    if rising_edge(clk) then
      if resetn = '0' then -- TODO: replace 'x' by the correct value
        y_vector <= (others => '0');
        cout <= '0'; -- fixed December 2024
      elsif en = '1' then -- TODO: replace 'x' by the correct value
        temp := cin;
        repeat:
          for I in 0 to N-1 loop
            y_vector(I) <= a_vector(I) xor b_vector(I) xor temp;
            temp := (a_vector(I) and b_vector(I)) or (a_vector(I)
              and temp) or (b_vector(I) and temp);
          end loop;
          cout <= temp;
        end if;
      end if;
    end process;
  end architecture for_loop;

```
